

A CAPIF-Aware AI-Native Telco Orchestration Platform for Intent-to-API Planning Across CAMARA and NEF Services

Türkan Doğa Durak
CTTC Internship Project
Barcelona, Spain
turkan.doga.durak@example.com

Abstract—This paper presents the design and implementation of a mini telco orchestration platform that combines Common API Framework (CAPIF) integration, CAMARA-aligned service semantics, NEF-backed telecom realization, and large language model (LLM)-assisted intent planning. The system is implemented as a FastAPI-based control plane with a product-style operator interface, embedded OpenAPI console, live CAPIF onboarding and discovery, provider publication, persistent NEF data services, and guarded orchestration execution. The platform supports both deterministic and LLM-assisted planning modes and validates generated API payloads before execution. A live Gemini-backed orchestration test confirms that natural-language service requests can be translated into structured QoD execution plans under runtime grounding and deterministic validation. The resulting system provides a practical baseline for reliability-aware intent-to-API orchestration research in 5G and beyond-5G service exposure environments.

Index Terms—CAPIF, CAMARA, NEF, 5G service exposure, intent-based orchestration, LLM planning, telecom APIs, OpenCAPIF

I. INTRODUCTION

Programmable network exposure is becoming a central design principle in modern telecom systems. CAPIF provides a standardized governance and discovery plane for exposed APIs, CAMARA defines developer-facing service semantics, and NEF-based providers bridge those interfaces to telecom data and network capabilities. At the same time, large language models introduce a new opportunity: translating high-level intents into structured API workflows without requiring the operator to manually compose payloads and endpoint sequences.

This project implements a working telco orchestration platform at that intersection. Rather than remaining at the conceptual architecture level, the system integrates live OpenCAPIF onboarding and discovery, live provider publication, live QoD and Location Retrieval execution, an embedded operator interface, and an LLM-assisted planning layer protected by deterministic validation.

The platform was developed not as an isolated mockup, but as a usable product-style control surface for demonstrating and evaluating end-to-end telecom orchestration behavior. This makes it suitable both for technical demonstrations and for

thesis-oriented experimentation on intent-to-API translation reliability.

II. SYSTEM OBJECTIVES AND CONTRIBUTIONS

The platform was built with four primary objectives:

- to unify CAPIF onboarding, discovery, and publication inside one operator-facing control plane,
- to expose CAMARA-aligned telecom capabilities through a live orchestration backend,
- to support both deterministic and LLM-assisted planning for natural-language requests,
- to ensure that execution remains guarded by deterministic validation and schema enforcement.

Its main contributions are:

- a working CAPIF-aware telco orchestration backend,
- a pluggable LLM planning layer grounded by local service metadata and CAPIF discovery,
- a live provider integration across QoD, Location Retrieval, and NEF Monitoring Event,
- a persistent and automatically seeded NEF data layer,
- a product-style front end with an embedded API console for live demonstrations.

III. PLATFORM ARCHITECTURE

The implemented system follows a layered control-plane architecture composed of user interaction, orchestration, planning, exposure governance, provider execution, and persistence layers.

A. High-Level Components

B. Repository Structure

IV. BACKEND ORCHESTRATION DESIGN

A. Core API Surface

The backend exposes the control and orchestration endpoints listed in Table III.

TABLE I
IMPLEMENTED PLATFORM COMPONENTS

Component	Responsibility
Operator Interface	Product-style landing page with embedded OpenAPI console and orchestration access
FastAPI Backend	Route handling, orchestration control, validation, and execution dispatch
Intent Engine	Deterministic parsing of user requests into structured execution steps
LLM Planner	Gemini-based structured planning under catalog and CAPIF grounding
CAPIF Client	Invoker onboarding, service discovery, and provider publication
Provider Layer	QoD, Location Retrieval, and NEF Monitoring Event service realization
Persistence Layer	MongoDB-backed NEF demo data with automatic initialization and persistence

TABLE II
KEY REPOSITORY PATHS

Path	Role
backend/main.py	FastAPI endpoint and primary route registration
backend/intent_engine.py	Deterministic intent parsing and execution-step construction
backend/llm_planner.py	LLM-assisted planning and grounding logic
backend/capif/client.py	CAPIF register, onboarding, discovery, and publish client
backend/models.py	Request, response, and payload schemas
frontend/index.html	Product-style operator landing page with embedded API console
providers/NEF/	NEF Monitoring Event stack and Mongo persistence
providers/CamaraLocationRetrieval	CAMARA Location Retrieval service
providers/camara-qod	CAMARA Quality-on-Demand service
docker-compose.capif	Main deployment attached to the OpenCAPIF network

B. Supported Telecom Capabilities

The current platform scope includes four orchestration targets:

- **Quality on Demand (QoD)** for temporary QoS session creation,
- **QoS Profiles** for profile inspection,
- **QoS Provisioning** for long-lived QoS assignment,
- **Location Retrieval** for device location access through NEF-backed monitoring.

C. Planning Modes

The POST `/orchestrate` endpoint supports three planning modes:

- **deterministic**: always uses the rule-based planner,
- **llm-assisted**: requires the LLM planner and fails explicitly if planning fails,
- **auto**: attempts the LLM planner first and falls back to deterministic planning if needed.

TABLE III
MAIN BACKEND ENDPOINTS

Method	Path	Purpose
GET	<code>/health</code>	Runtime health and status summary
GET	<code>/catalog/service</code>	Local CAMARA-aligned capability catalog
POST	<code>/orchestrate</code>	Intent-to-execution planning endpoint
GET	<code>/capif/status</code>	CAPIF and register connectivity check
GET	<code>/capif/invoker</code>	Stored invoker identity details
POST	<code>/capif/invokers</code>	Live invoker onboarding against CAPIF
GET	<code>/capif/discover</code>	Live service discovery through CAPIF
POST	<code>/capif/publish</code>	Provider publication endpoint
POST	<code>/capif/publish/publication</code>	Publication of catalog services into CAPIF

This design keeps the AI layer optional yet operational, which is particularly useful for both live demos and controlled experimentation.

V. CAPIF INTEGRATION WORKFLOW

A. CAPIF Roles in the Platform

CAPIF provides the exposure governance plane used by the system. The key roles are:

- **Invoker**: the mini orchestration platform or provider client consuming exposed APIs,
- **Provider / AEF**: the service implementation exposing an API through CAPIF,
- **CAPIF Core**: the OpenCAPIF services managing discovery and onboarding,
- **Register**: the bootstrap service used for identity and credential preparation.

B. Invoker Onboarding Flow

The implemented onboarding flow in `backend/capif/client.py` performs the following steps:

- 1) authenticate against the register service,
- 2) ensure a demo user exists,
- 3) obtain credential material for that user,
- 4) generate a local private key and certificate signing request,
- 5) submit the onboarding request to CAPIF Core,
- 6) receive an `apiInvokerId` and invoker certificate,
- 7) store runtime invoker details for later discovery and execution.

This workflow enables the platform to operate as a real CAPIF invoker rather than as a static mock client.

C. Discovery Flow

The discovery workflow is implemented by GET `/capif/discover`:

- 1) read the stored invoker identity,

- 2) extract the invoker identifier,
- 3) query the CAPIF discovery endpoint,
- 4) return live service descriptions from the registry.

In the current platform deployment, discovery returns published services including QoD, Location Retrieval, NEF Monitoring Event, and the default OpenCAPIF demo service.

D. Provider Publication Flow

Provider publication follows the standard sequence:

- 1) start the provider service,
- 2) prepare the OpenAPI descriptor,
- 3) onboard the provider identity if needed,
- 4) publish API metadata into CAPIF,
- 5) verify that the API appears in CAPIF discovery results.

VI. CAMARA AND NEF SERVICE REALIZATION

A. Role of CAMARA

CAMARA defines the exposed service semantics consumed by the orchestration platform. In this system:

- CAPIF governs onboarding and discovery,
- CAMARA defines the API-facing service contracts,
- provider services bridge these APIs to telecom behavior and data.

B. Role of NEF

The NEF Monitoring Event provider acts as the telecom-facing realization layer for location workflows. It:

- receives a monitoring request,
- maps phone numbers to IMSI identifiers,
- reads location information from MongoDB,
- enriches responses with polygonal area information,
- returns a 3GPP-style monitoring result that upstream services can transform.

C. Location Retrieval End-to-End Flow

The live Location Retrieval path executes as follows:

- 1) the client calls the Location Retrieval provider,
- 2) the provider constructs a 3GPP Monitoring Event subscription,
- 3) the provider obtains CAPIF security context and access tokens,
- 4) the provider calls the NEF Monitoring Event API,
- 5) the NEF resolves identifiers and reads location plus polygon data,
- 6) the response is normalized into CAMARA Location Retrieval format,
- 7) the final location response is returned to the caller.

D. QoD Flow

The QoD path is simpler:

- 1) the user intent is mapped to the qod capability,
- 2) the planner generates a session creation payload,
- 3) the QoD provider handles the session request,
- 4) the provider result is returned or validated in dry-run mode.

VII. PERSISTENCE AND RELIABILITY ENGINEERING

A. Persistent NEF Data

During earlier integration stages, the NEF stack lost location and polygon records after recreation. This created operational instability for the Location Retrieval chain. The issue was resolved through:

- a named Docker volume mounted at `/data/db`,
- an automatic Mongo initialization script mounted at `/docker-entrypoint-initdb.d/mongo-init.js`.

The resulting behavior is:

- 1) the first deployment seeds the required data automatically,
- 2) standard restarts preserve the existing dataset,
- 3) the NEF-backed location pipeline survives stack recreation without manual reseeding.

B. Validation Guardrails

Execution never depends solely on LLM output. All generated payloads are validated using backend-side schemas and structured execution models. This ensures:

- correct field names,
- correct structural nesting,
- presence of required values,
- deterministic failure before provider invocation if a payload is malformed.

This separation between probabilistic planning and deterministic enforcement is a central architectural property of the platform.

VIII. LLM-ASSISTED PLANNING LAYER

A. Planning Strategy

The file `backend/llm_planner.py` implements the AI planning path. It follows this procedure:

- 1) load the local service catalog,
- 2) optionally incorporate live CAPIF discovery results,
- 3) build a constrained prompt containing allowed services, operations, and payload expectations,
- 4) call the configured model provider,
- 5) parse the returned JSON plan,
- 6) transform the output into an internal execution step,
- 7) pass the result into deterministic validation.

The planner therefore proposes structured actions but never executes them directly.

B. Live Gemini Orchestration Result

After introducing a new Gemini project and API key, the LLM-assisted path was validated with a live end-to-end request using the intent:

Boost video QoS for +905551112233 for 15 minutes.

The request was submitted to `POST /orchestrate` in `llm-assisted` mode with `dry_run=true`. The backend returned HTTP 200 and produced the following execution semantics:

- selected service: `qod`,

- selected operation: `createSession`,
- target path: `/quality-on-demand/vwip/sessions`,
- device identifier: `+905551112233`,
- duration: 900 seconds,
- QoS profile: `QOS_E_STREAMING`.

This confirms that the LLM planner now succeeds as a live runtime component rather than acting only as a fallback candidate under external quota failures.

C. Productized Front End

To support direct presentation and operation, the front end was also converted from a demo-style landing page into a product-style control surface. The revised interface:

- removes presentation-specific or informal wording,
- presents the system as a telecom control plane,
- embeds the live API console directly in the primary workspace,
- provides a concise operator workflow for onboarding, discovery, and orchestration.

IX. OPERATOR WORKFLOW AND API CONSOLE

The intended live presentation flow is:

- 1) open the landing page at `http://localhost:8000/`,
- 2) verify backend health and CAPIF connectivity,
- 3) inspect the local service catalog,
- 4) perform invoker onboarding,
- 5) review discovered services through CAPIF,
- 6) submit orchestration requests through the embedded API console.

Representative orchestration bodies include:

- `{"intent": "Boost video QoS for +905551112233 for 15 minutes", "dry_run": true, "orchestration_mode": "llm-assisted"}`,
- `{"intent": "Retrieve the latest location for +905551112233", "dry_run": true, "orchestration_mode": "llm-assisted"}`,
- `{"intent": "Find the best available QoS profile for +905551112233", "dry_run": true, "orchestration_mode": "auto"}`.

X. CURRENT STATUS AND NEXT STEPS

At the current stage, the platform provides:

- live CAPIF invoker onboarding,
- live CAPIF service discovery,
- live provider publication visibility through CAPIF,
- live QoS and NEF-backed Location Retrieval service paths,
- persistent NEF data with automatic initialization,
- a working Gemini-backed LLM planner,
- a product-style operator interface with an embedded API console.

The next logical extensions include richer policy validation, additional CAMARA services, telemetry and evaluation instrumentation, and a formal reliability scoring layer for structured intent-to-API translation experiments.

XI. CONCLUSION

This paper presented a working AI-native telco orchestration platform that integrates CAPIF governance, CAMARA service semantics, NEF-backed telecom realization, deterministic validation, and LLM-assisted planning into one deployable control plane. The system goes beyond mock orchestration by supporting live onboarding, discovery, provider publication, and execution-ready planning. The successful Gemini-backed orchestration result demonstrates that natural-language telecom requests can be translated into structured API actions under explicit runtime constraints. As a result, the platform forms a practical baseline for future work on trustworthy intent-driven service exposure in 5G and beyond-5G environments.